

## Session 7

# Ensemble Methods

*Why one tree is never enough —  
and how to build a forest that's far more reliable than any single tree*

### Bagging

Reduce variance  
with parallel trees

### Random Forest

Diverse trees →  
better predictions

### Boosting

Correct errors  
sequentially

# The Problem: Decision trees are often instable

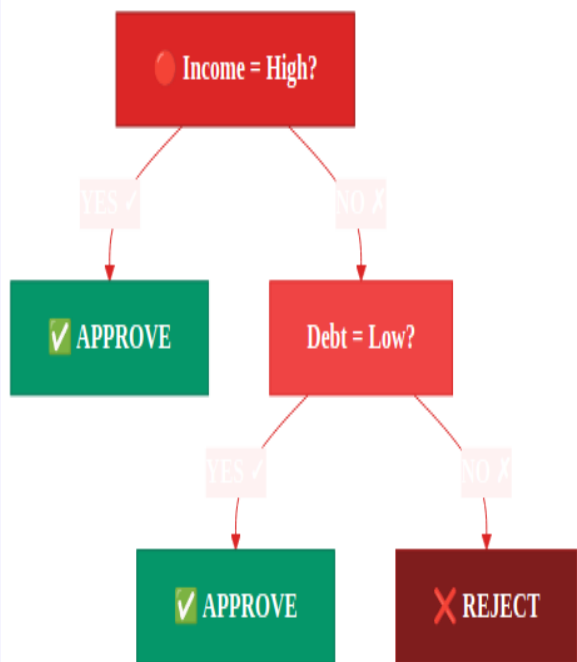
WHY — Business Intuition

BankEU — 10 Applications

| #  | Income | Debt | Age   | Loan? |
|----|--------|------|-------|-------|
| 1  | High   | Low  | Old   | YES   |
| 2  | Low    | High | Young | NO    |
| 3  | High   | Low  | Young | YES   |
| 4  | Low    | Low  | Old   | YES   |
| 5  | Low    | High | Old   | NO    |
| 6  | High   | High | Young | NO    |
| 7  | High   | Low  | Old   | YES   |
| 8  | Low    | Low  | Young | NO    |
| 9  | High   | High | Old   | YES   |
| 10 | Low    | High | Young | NO    |

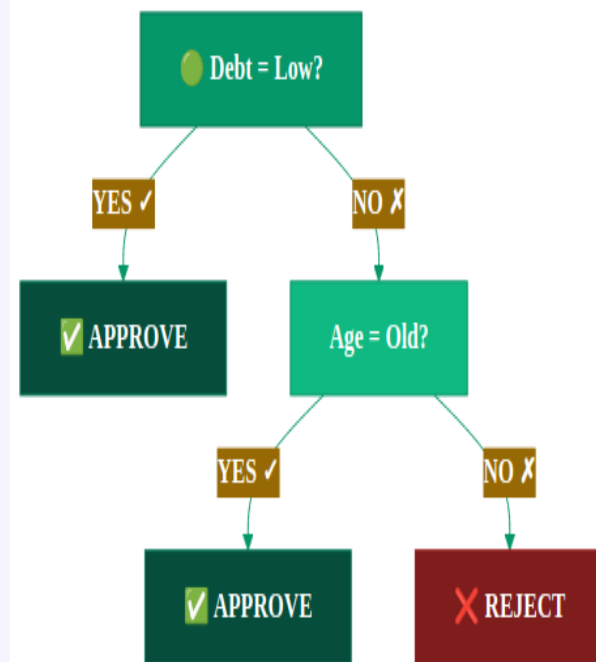
Tree A — trained on rows 1-8

Root split: Income = High?



Tree B — trained on rows 3-10

Root split: Debt = Low?



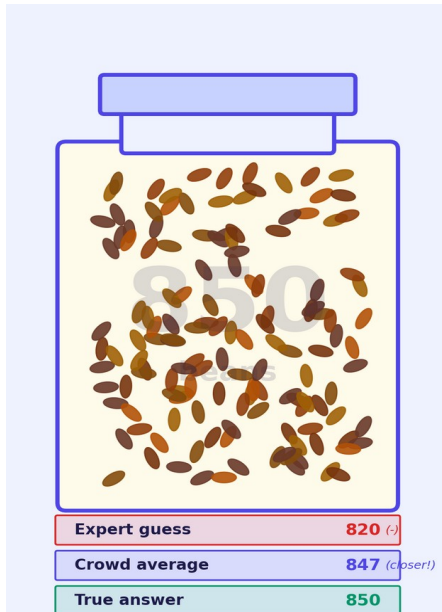
Remove just 2 rows → the ROOT split changes completely. This is HIGH VARIANCE — the core problem ensembles solve.

# The Core Idea: Wisdom of Crowds

Can many weak models beat one strong model?

WHY — Business Intuition

## The Jar of Beans



800 people each guess  
the number of beans.

Their average:

847

True answer:

850

Expert  
guess: 820

The crowd beats the expert — not because each person is smart,  
but because their errors cancel out.

## The Math (Condorcet's Theorem)

If each model has 60% accuracy and they vote independently:

3 models

$$P = 0.6^3 + 3(0.6^2)(0.4) = 0.648$$

64.8%

11 models

$$P(\text{majority}) = 75\%$$

75%

101 models

$$P(\text{majority}) = 98\%$$

98%

The ONE condition:  
Each model must make DIFFERENT errors.  
If they all fail on the same cases, averaging does nothing.

# Part 1: Bagging

*Bootstrap Aggregating — How to create diverse trees from one dataset*

Part of Session 7 — Ensemble Methods

# Why Bootstrap Sampling? The Business Motivation

*Before the math — why does this trick actually help?*

**The problem: we only have ONE dataset. We can't go collect more BankEU loan data.  
How do we train 100 different trees that all make DIFFERENT mistakes?**



## The Lottery Analogy

Imagine shuffling 10 loan applications and drawing 10 with replacement. Some applications appear twice — row 3 might be drawn on draw 1 AND draw 6. Some never appear.

Result: Each draw of 10 is a slightly different dataset.

**This is bootstrap sampling.**



## The Journalist Analogy

A journalist interviews 10 people about a policy. Two interviewers each talk to 10 people — but since they draw from the same pool, they each get a slightly different mix.

Result: Two journalists write different stories from the same population.

**Each tree = one journalist's story.**



## The BankEU Business Case

Your credit team has 1,000 historical loan decisions. Bootstrap lets you simulate 'what if we had slightly different data?'

Each simulated dataset trains a different tree. Their combined vote is more robust than any single model trained on your one real dataset.

**100 trees = 100 slightly different models.**

**Key: bootstrapping doesn't create new data — it creates VARIATION in training sets. That variation means trees make DIFFERENT errors, and different errors cancel out when we vote.**

# Step 1: Bootstrap Sampling — By Hand

Draw  $N$  samples *WITH* replacement from the original dataset

WHAT — Mathematics

| Original Dataset (N=10)  |
|--------------------------|
| 1 (High/Low/Old → YES)   |
| 2 (Low/High/Young → NO)  |
| 3 (High/Low/Young → YES) |
| 4 (Low/Low/Old → YES)    |
| 5 (Low/High/Old → NO)    |
| 6...                     |
| 7...                     |
| 8...                     |
| 9...                     |
| 10...                    |

Roll the dice



| Bootstrap Sample 1 |
|--------------------|
| Row 1 ★            |
| Row 1 ★            |
| Row 3              |
| Row 5 ★            |
| Row 5 ★            |
| Row 7              |
| Row 2              |
| Row 9              |
| Row 4              |
| Row 6              |

Row 1 appears TWICE. Row 8 never appears.

| Bootstrap Sample 2 |
|--------------------|
| Row 2              |
| Row 4              |
| Row 7 ★            |
| Row 7 ★            |
| Row 9              |
| Row 1              |
| Row 3              |
| Row 6              |
| Row 10             |
| Row 5              |

Row 7 appears TWICE. Row 8 never appears.

| Bootstrap Sample 3 |
|--------------------|
| Row 3              |
| Row 6              |
| Row 9              |
| Row 2 ★            |
| Row 2 ★            |
| Row 8              |
| Row 1              |
| Row 4              |
| Row 7              |
| Row 10             |

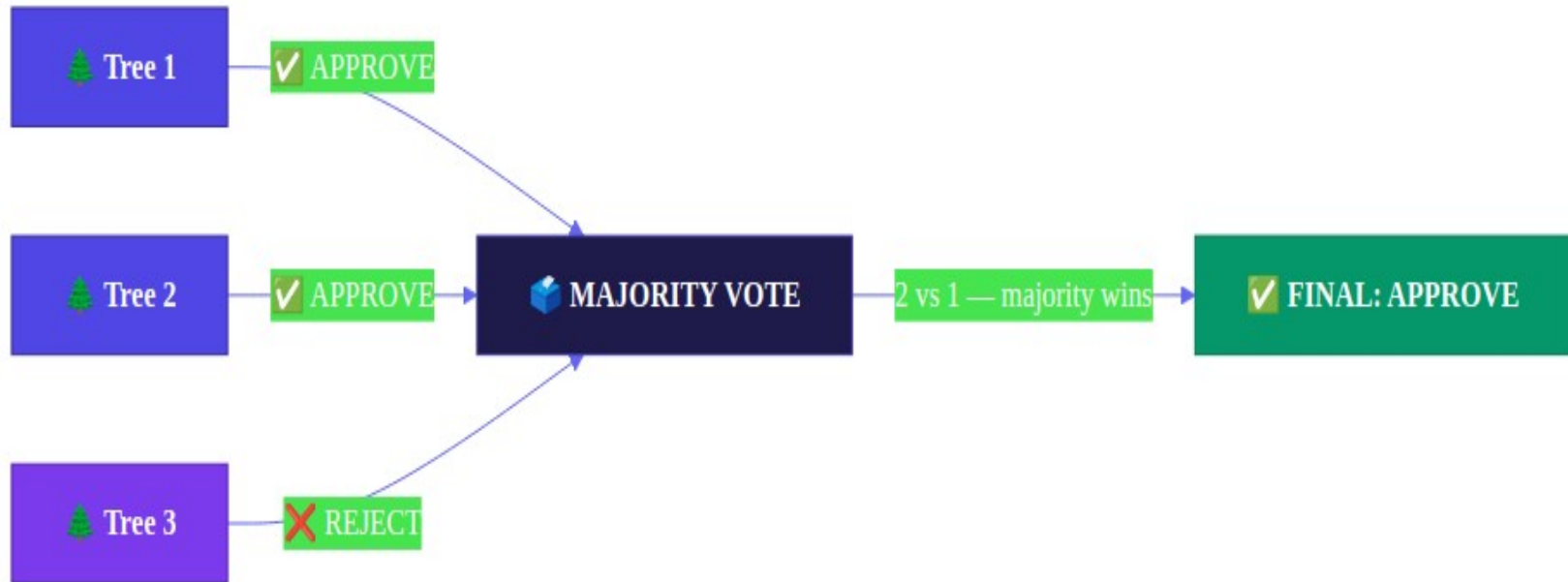
Row 2 appears TWICE. Row 5 never appears.

★ = duplicated rows | Each sample is size  $N=10$ , but has ~6-7 unique rows (~63.2%) and 3-4 repeated ones. Each tree trains on a different subset — so they'll make different errors!

# Step 2: Majority Vote — How Three Trees Decide

Each tree votes independently; the majority overrules any single wrong tree

New applicant to classify: Income = HIGH Debt = LOW Age = YOUNG → Approve or Reject?



## Tree 1 & Tree 2

Both trained on samples where High Income + Low Debt → approval. They vote APPROVE.



## Tree 3 (wrong)

Its bootstrap sample had many young rejections. It votes REJECT — but it's outvoted.



## Majority wins

2 vs 1: the wrong tree's error is corrected. This is the power of ensemble averaging.

# Implementing Bagging in sklearn

*BaggingClassifier: the explicit version before Random Forest*

HOW — Code

```
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score
from sklearn.datasets import load_breast_cancer
import numpy as np

X, y = load_breast_cancer(return_X_y=True)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# — Explicit Bagging —————
bag = BaggingClassifier(
    estimator=DecisionTreeClassifier(max_depth=None),
    n_estimators=100,          # number of trees
    max_samples=1.0,          # fraction of rows per tree
    max_features=1.0,         # ALL features (pure bagging)
    bootstrap=True,           # sample WITH replacement
    bootstrap_features=False,
    oob_score=True,           # free validation!
    n_jobs=-1,
    random_state=42
)

bag.fit(X_train, y_train)
print(f"OOB score: {bag.oob_score_:.3f}")
print(f"Test score: {bag.score(X_test, y_test):.3f}")

scores = cross_val_score(bag, X, y, cv=5, scoring='roc_auc')
print(f"CV AUC: {scores.mean():.3f} ± {scores.std():.3f}")
```

## **estimator=DecisionTreeClassifier**

Any sklearn estimator works here — SVM, KNN, even LogReg. Bagging is not tree-specific. Trees are just the most common choice because they're fast and high-variance.

## **max\_features=1.0 (all features)**

This is the key difference from Random Forest. Bagging uses ALL features at every split. RF will use only  $\sqrt{p}$  features. This is why RF creates more diverse trees.

## **bootstrap=True**

Each tree trains on a random sample drawn WITH replacement. Set bootstrap=False to sample without replacement — that's called Pasting, not Bagging.

## **oob\_score=True**

Each tree doesn't see ~37% of rows. Those rows form the Out-Of-Bag set. Evaluating on OOB gives a free validation score — no cross-validation needed.



# Why Do Bootstrap Samples Have ~63.2% Unique Rows?

A useful mathematical fact — explained from scratch

WHAT — Mathematics

## Step-by-step Derivation

1 Dataset has  $N=10$  rows. We draw 10 times with replacement.

2 Probability that row #1 is NOT picked on draw 1:  $9/10 = 0.9$

3 Probability that row #1 is missed on ALL 10 draws:  
 $(9/10)^{10} = 0.9^{10} \approx 0.349$

4 So probability row #1 IS included:  
 $1 - 0.349 = 0.651 \approx 65\%$

5 As  $N \rightarrow \infty$ :  $1 - (1 - 1/N)^N \rightarrow 1 - 1/e \approx 0.632$

## What This Means in Practice

**$\approx 63\%$**

of rows appear in each bootstrap sample (the training set)

**$\approx 37\%$**

of rows are left out (called Out-of-Bag or OOB rows)

**Free!**

OOB rows act like a built-in validation set — often reducing the need for cross-validation during tuning, though a held-out test set remains best practice.

**Diverse**

Each sample has different OOB rows, so each tree sees a different 'test set'

**OOB Score in sklearn: `RandomForestClassifier(oob_score=True)` → free validation at no extra cost!**

# OOB Score: Built-in Validation for Random Forest

The 37% of rows each tree never saw become its personal test set

## How OOB Works — Step by Step

1

### Bootstrap creates each training set

Tree 42 is trained on rows: {1,1,3,5,6,6,8,9,9,10}.  
Rows 2, 4, 7 never appeared → these are OOB for Tree 42.

2

### OOB rows are used for evaluation

Tree 42 predicts row 2 (which it never trained on).  
This is a genuine out-of-sample prediction.

3

### Every row is OOB for ~37% of trees

Row 2 is OOB for Tree 42, Tree 15, Tree 67...  
Each row gets evaluated by ~37 trees out of 100.

4

### Aggregate all OOB predictions

For each row, average the predictions from the trees that didn't see it.  
Compare to true label → OOB score.

## OOB Score in Practice

| Metric        | OOB                                          | 5-Fold CV                                          |
|---------------|----------------------------------------------|----------------------------------------------------|
| Accuracy      | 0.961                                        | 0.958                                              |
| Training time | 1 forest fit                                 | ~5 model fits                                      |
| Data used     | ~63%/tree; all rows contribute across forest | 80% train / 20% validate; every row validated once |
| Reliability   | Good for bagging (N>500, trees≥100)          | More general; robust across model types            |

### When to trust OOB score

- ✓ N > 500 rows — enough OOB samples per tree
- ✓ n\_estimators ≥ 100 — stable OOB estimate

### sklearn usage:

```
rf = RandomForestClassifier(oob_score=True)
rf.fit(X_train, y_train)
print(rf.oob_score_)    # e.g. 0.961
```

# Part 2: Random Forest

*Bagging + one extra trick that changes everything*

Part of Session 7 — Ensemble Methods

# The Problem with Plain Bagging: Correlated Trees

Bagging alone doesn't create enough diversity

## The Exam Panel Analogy

### Bad Panel (Bagging alone)



You hire 5 interviewers, all from the same school, all with the same specialty. They all like the same candidates and all hate the same ones. Majority vote adds nothing — they always agree.

### Good Panel (Random Forest)



You hire 5 interviewers from different backgrounds: finance, tech, HR, strategy, operations. They notice different things. Now the majority vote is genuinely informative.

*In ML: if Income is the strongest feature, ALL trees will use Income as their root split. They'll be almost identical — and majority vote won't help.*

## Random Forest Solution

At each split, each tree can only consider a RANDOM SUBSET of features.

Our dataset has 3 features:

- Income
- Debt
- Age

With  $\sqrt{p}$  rule ( $\sqrt{3} \approx 1.2$ ):

Tree 1 sees: Income, Debt

Tree 2 sees: Debt, Age

Tree 3 sees: Income, Age

Now they MUST split differently!

Each notices different patterns.

$\sqrt{p}$  for classification |  $p/3$  for regression

# Why Does Random Forest Outperform a Single Tree?

The business case for using 100 trees instead of one

BankEU scenario: you trained ONE decision tree. It gets 78% accuracy.  
Your manager asks: 'Can we do better without hiring a data scientist for weeks?'

## Single Decision Tree

### Strengths:

- ✓ Fast to train
- ✓ Fully interpretable
- ✓ Visualisable

### Weaknesses:

- ✗ Unstable — change 3 rows → new tree
- ✗ Overfits training data easily
- ✗ One bad split ruins everything

Use when stakeholders demand explanation (e.g. EU AI Act audit). Otherwise, upgrade.

## Bagging (100 trees)

### Strengths:

- ✓ More stable than 1 tree
- ✓ OOB score = free validation
- ✓ Any base estimator works

### Weaknesses:

- ✗ All trees can split on same feature first
- ✗ Correlated trees = less diversity
- ✗ Slightly harder to explain

Better than one tree. But trees are still too similar. We can do better.

## Random Forest

### Strengths:

- ✓ Diverse trees → uncorrelated errors
- ✓ OOB score built in
- ✓ Feature importance for free

### Weaknesses:

- ✗ Slower than 1 tree
- ✗ Often outperformed by tuned gradient boosting on structured/tabular data
- ✗ Still not great for very noisy data

The sweet spot: 5 minutes to train, 87%+ accuracy, interpretable importance scores.

# Random Forest: Full Algorithm

Bagging + random feature subsets = diverse, stable forest

1

## Bootstrap

Draw N samples with replacement from training data

→ 63.2% unique, 36.8% OOB

2

## Random Features

At each node split, randomly select  $\sqrt{p}$  features to consider

→ forces diversity between trees

3

## Grow Full Tree

Train unpruned decision tree on the bootstrap sample

→ each tree has low bias, high variance

4

## Repeat 1-3

Grow  $n_{\text{estimators}}$  trees (typically 100-500)

→ more trees = more stable; test error usually plateaus rather than worsens

5

## Aggregate

Classification: majority vote | Regression: average prediction

→ variance cancels out across trees

Key params:  $n_{\text{estimators}}$  ↑ = more stable (not overfit) |  $\text{max\_features}$  controls diversity |  $\text{min\_samples\_leaf}$  controls tree depth

## sklearn

```
from sklearn.ensemble import (
    RandomForestClassifier
)

rf = RandomForestClassifier(
    n_estimators=100,
    max_features="sqrt",
    oob_score=True,
    n_jobs=-1,
    random_state=42
)

rf.fit(X_train, y_train)

# Free OOB validation!
print(rf.oob_score_)

# Feature importance
rf.feature_importances_
```

# Random Forest: Feature Subsets in Action

Tracing exactly how two trees split differently on the same node

BankEU dataset: 3 features (Income, Debt, Age).  $\sqrt{3} \approx 1.7 \rightarrow$  each node considers 2 randomly selected features.  
Let's trace the ROOT NODE split for Tree 1 and Tree 2 from scratch.

## Tree 1 — Root Node

Features available at this node:

✓ Income

✓ Debt

✗ Age

(random subset)

Now evaluate each allowed feature:

### Split on Income = High?

Left (High): 4 YES, 1 NO  $\rightarrow$  Gini =  $1 - (4/5)^2 - (1/5)^2 = 0.32$   
Right (Low): 1 YES, 4 NO  $\rightarrow$  Gini =  $1 - (1/5)^2 - (4/5)^2 = 0.32$   
Weighted:  $(5/10) \times 0.32 + (5/10) \times 0.32 = 0.32$

### Split on Debt = Low?

Left (Low): 5 YES, 1 NO  $\rightarrow$  Gini =  $1 - (5/6)^2 - (1/6)^2 = 0.28$   
Right (High): 0 YES, 4 NO  $\rightarrow$  Gini = 0  
Weighted:  $(6/10) \times 0.28 + (4/10) \times 0 = 0.167$

$\rightarrow$  Tree 1 splits on DEBT (Gini 0.167 < 0.32) — Age was never even considered

## Tree 2 — Root Node

Features available at this node:

✗ Income

✓ Debt

✓ Age

Now evaluate each allowed feature:

### Split on Debt = Low?

Left (Low): 5 YES, 1 NO  $\rightarrow$  Gini = 0.28  
Right (High): 0 YES, 4 NO  $\rightarrow$  Gini = 0  
Weighted Gini = 0.167

### Split on Age = Old?

Left (Old): 3 YES, 1 NO  $\rightarrow$  Gini = 0.375  
Right (Young): 2 YES, 4 NO  $\rightarrow$  Gini = 0.44  
Weighted:  $(4/10) \times 0.375 + (6/10) \times 0.44 = 0.414$

$\rightarrow$  Tree 2 also splits on DEBT (0.167 beats 0.414) — Income was never considered

# Feature Importance: How Does Random Forest Know What Matters?

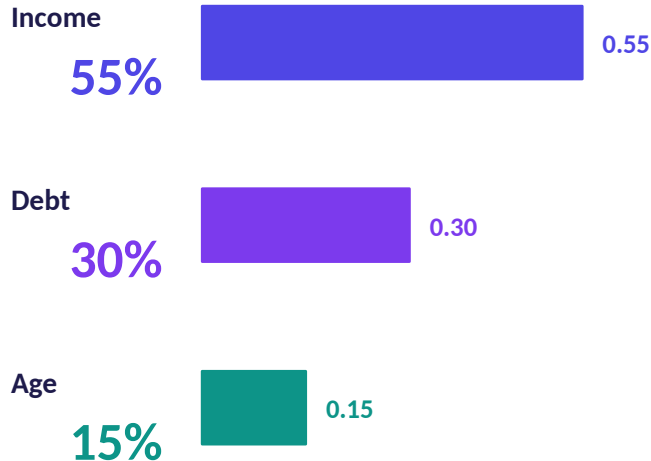
Mean Decrease in Impurity — explained step by step

## How It's Calculated (MDI)

- 1 For each tree, look at every split made on feature F.
- 2 Calculate how much that split reduced impurity (Gini / Entropy)  $\times$  the number of samples it affected.
- 3 Sum this across all trees.
- 4 Normalize so all importances sum to 1.0.

*Interpretation: Feature importance = 0.42 means this feature reduced impurity by 42% of the total reduction across all trees and all splits.*

## Example: BankEU Loan Model



### Interpretation:

*"Income accounts for 55% of the total impurity reduction. Age barely*

**⚠** MDI is biased toward high-cardinality features (many unique values). Use Permutation Importance (`sklearn's permutation_importance()`) for more reliable results, especially with categorical data.



# Part 3: Boosting

*A completely different strategy — learn from your mistakes, sequentially*

Part of Session 7 — Ensemble Methods

# Boosting: Sequential Error Correction

Each model focuses on the mistakes the previous model made

## The Tutor-Student Analogy

|        |                                                                          |
|--------|--------------------------------------------------------------------------|
| Week 1 | Student takes test → 60% score. Tutor notes which 4 problems were WRONG. |
| Week 2 | New study plan focused entirely on those 4 wrong topics. Score: 75%.     |
| Week 3 | 3 new errors appear. Tutor focuses again on only those. Score: 85%.      |
| Final  | All study sessions combined = the full knowledge. Score: 92%.            |

Bagging: parallel, independent trees → reduces VARIANCE  
Boosting: sequential, each fixes last one's errors → reduces BIAS

## The Algorithm



# AdaBoost — Step by Step (Round 1)

6 data points, starting weights = 1/6 each

Simplified dataset: 6 applicants | + = Approve | - = Reject

| # | Income | Debt | Label | Weight (Round 1) | Tree 1 Predicts | Correct? |
|---|--------|------|-------|------------------|-----------------|----------|
| 1 | High   | Low  | +     | 1/6 = 0.167      | +               | ✓        |
| 2 | Low    | High | -     | 1/6 = 0.167      | -               | ✓        |
| 3 | High   | High | +     | 1/6 = 0.167      | +               | ✓        |
| 4 | Low    | Low  | -     | 1/6 = 0.167      | +               | X WRONG  |
| 5 | High   | Low  | +     | 1/6 = 0.167      | +               | ✓        |
| 6 | Low    | High | -     | 1/6 = 0.167      | +               | X WRONG  |

After Round 1: Wrong predictions (rows 4, 6) get HIGHER weights → next tree must focus on them!

→ See next slide for Round 2 and how weight updates change the focus

## Round 1 Calculations

**Error Rate ( $\epsilon$ )**  
 $\epsilon$  = sum of weights of wrong predictions  
 $= 0.167 + 0.167 = 0.333$

**Model Weight ( $\alpha$ )**  
 $\alpha = 0.5 \times \ln((1-\epsilon)/\epsilon)$   
 $= 0.5 \times \ln(2) \approx 0.347$

**Update Weights (wrong cases)**  
 $w_{\text{new}} = w_{\text{old}} \times e^{\alpha}$   
 $= 0.167 \times e^{0.347} \approx 0.236$

**Update Weights (correct cases)**  
 $w_{\text{new}} = w_{\text{old}} \times e^{(-\alpha)}$   
 $= 0.167 \times e^{(-0.347)} \approx 0.118$

**Normalize**  
Divide all weights so they sum to 1.0

# AdaBoost Round 2: How Weights Change Training

The tree doesn't just 'notice' hard cases — weights change the Gini calculation

Updated weights after Round 1: Rows 4 & 6 (wrong) → 0.236 each | Rows 1,2,3,5 (correct) → 0.118 each | Total = 1.0

Round 2 Dataset with NEW Weights

| # | Income | Debt | Label | Weight | Visual weight |
|---|--------|------|-------|--------|---------------|
| 1 | High   | Low  | +     | 0.118  | ■             |
| 2 | Low    | High | -     | 0.118  | ■             |
| 3 | High   | High | +     | 0.118  | ■             |
| 4 | Low    | Low  | -     | 0.236  | ■ ← was WRONG |
| 5 | High   | Low  | +     | 0.118  | ■             |
| 6 | Low    | High | -     | 0.236  | ■ ← was WRONG |

## Round 2 Final Prediction:

Tree 2 rule: Income=High → +, Income=Low → -

Tree 2 error: rows 2,3 (Low/+ and High/-)

$$\epsilon_2 = 0.118 + 0.118 = 0.236$$

$$\alpha_2 = 0.5 \times \ln((1-0.236)/0.236) \approx 0.583$$

$\alpha_2 > \alpha_1$  (0.583 > 0.347) → Tree 2 gets MORE vote weight!

## Weighted Gini at Root Node

The key insight: Gini is now computed using WEIGHTS, not counts.  
A misclassified row with weight 0.236 counts more than a correct row with 0.118.

### Try split: Income = High?

Left branch (Income=High): rows 1,3,5

Weights: 0.118, 0.118, 0.118 → sum = 0.354

Labels: +, +, + (all positive)

Weighted Gini = 0 (pure!)

Right branch (Income=Low): rows 2,4,6

Weights: 0.118, 0.236, 0.236 → sum = 0.59

+ weight: 0 (none) - weight: 0.59

Weighted Gini = 0 (pure!)

Weighted Gini = 0. Tree 2 splits on Income = perfect pure split!

### Why did this happen?

Rows 4 & 6 (Low income, wrong in Round 1) now carry double the weight.  
Tree 2 'sees' them as much more important and builds a rule that correctly classifies them.

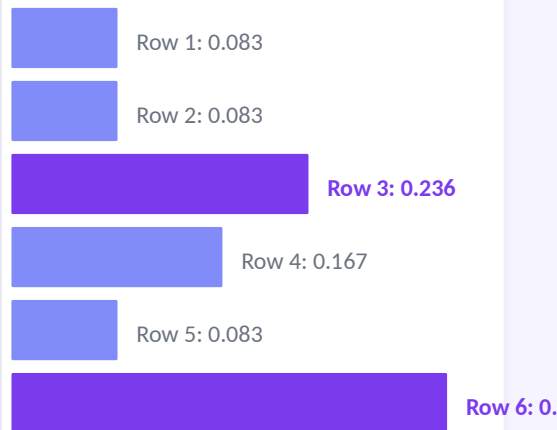
After this round, rows 4 & 6 will have their weights REDUCED again.

# AdaBoost — Weight Evolution Across Rounds

*Wrong predictions grow in importance until they're corrected*

| How weights change across 3 rounds |       |                    |                       |                           |                |
|------------------------------------|-------|--------------------|-----------------------|---------------------------|----------------|
| Row                                | Label | Round 1<br>(equal) | Round 2<br>(wrongs ↑) | Round 3<br>(new wrongs ↑) | Final<br>Focus |
| 1                                  | +     | 0.167              | 0.118                 | 0.083                     | Low            |
| 2                                  | -     | 0.167              | 0.118                 | 0.083                     | Low            |
| 3                                  | +     | 0.167              | 0.118                 | 0.236                     | HIGH           |
| 4                                  | -     | 0.167              | 0.236                 | 0.167                     | Med            |
| 5                                  | +     | 0.167              | 0.118                 | 0.083                     | Low            |
| 6                                  | -     | 0.167              | 0.236                 | 0.347                     | VERY HIGH      |

Weight Visualization  
(Round 3)



## AdaBoost Formula Summary

Final prediction =  $\text{sign}(\alpha_1 \cdot h_1(x) + \alpha_2 \cdot h_2(x) + \alpha_3 \cdot h_3(x))$

Where:  $\alpha_i$  = model weight (higher  $\alpha$  = better model got more say)

$h_i(x)$  = weak model  $i$ 's prediction (+1 or -1)

$\text{sign}()$  converts the sum to a class label

KEY: A model with lower error gets a HIGHER  $\alpha$  (more vote weight). As error  $\rightarrow 0$ ,  $\alpha$  grows very large. A model at random chance (50%) gets  $\alpha = 0$ .

# Why Gradient Boosting? When Random Forest Isn't Enough

Moving from variance reduction to bias reduction — a different problem

You've deployed Random Forest on BankEU. AUC = 0.84. Your manager says the regulator wants 0.90+. RF with 500 trees gives 0.845. Adding more trees doesn't help. You've hit RF's ceiling.

## Why Random Forest Has a Ceiling

### All trees are equally weighted

A tree that gets 70% accuracy votes the same as one that gets 90%. Good and bad trees cancel each other out less efficiently.

### Trees are trained independently

Tree 50 doesn't know Tree 49 already handles high-income cases well. It wastes capacity redoing the same work.

### Averaging reduces variance, not bias

If ALL trees are slightly wrong in the same direction (systematic bias), averaging doesn't fix it — it just gives you the average wrong answer.

## How Gradient Boosting Fixes This

### Each tree targets the RESIDUAL errors

Tree 2 doesn't learn the whole problem — it only learns to predict what Tree 1 got WRONG. Maximum efficiency: no repeated work.

### Each tree contributes an additive correction

Each new tree is scaled by a learning rate  $\eta$ . No voting — just additive corrections, each one reducing the remaining error by a small step.

### Sequential: each tree makes the previous smarter

After 200 trees, the ensemble has specialised models for every tricky region of the data. RF's 200 trees all try to solve the whole problem.

Rule of thumb: RF first (fast, stable baseline) → if you need more accuracy: XGBoost → if you need speed on big data: LightGBM

# Gradient Boosting: Fit the Residuals

Each tree corrects the ERRORS (residuals) of all previous trees

## Worked Example: Predict loan amount (€) from income level

| Row | Income | True Amount (€) | Round 0 (avg = 23k) | Residual 1 (true - pred) | Tree 2 Predicts | New Pred (R0 + T2) |
|-----|--------|-----------------|---------------------|--------------------------|-----------------|--------------------|
| 1   | Low    | 15k             | 23k                 | -8k                      | -7k             | 16k                |
| 2   | Medium | 22k             | 23k                 | -1k                      | 0k              | 23k                |
| 3   | Medium | 25k             | 23k                 | +2k                      | +2k             | 25k                |
| 4   | High   | 28k             | 23k                 | +5k                      | +6k             | 29k                |
| 5   | High   | 35k             | 23k                 | +12k                     | +11k            | 34k                |

## What's happening?

1. Round 0: Predict the mean (€23k) for everyone
2. Calculate residuals = errors
3. Tree 2 learns to predict those RESIDUALS
4. New prediction = old +  $\eta \times$  tree2 (this example uses  $\eta=1$  for clarity)
5. Repeat — each tree reduces remaining error

Algorithm:  $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$

Where  $\eta$  = learning rate (typically 0.01–0.3 in practice;  $\eta=1$  above for illustration),  $h_m$  = new tree fitted on residuals,  $F_m$  = updated prediction

# Why Is It Called GRADIENT Boosting?

Residuals are not arbitrary — they are the gradient of the loss function

## Building the Connection

1

### Start with a Loss Function (MSE)

We want to minimise:  $L(y, F) = \frac{1}{2}(y - F)^2$

Where  $y$  = true value,  $F$  = our current prediction.  
(The  $\frac{1}{2}$  is just for clean derivatives.)

2

### Compute the Gradient of $L$ w.r.t. $F$

$$\partial L / \partial F = \partial [\frac{1}{2}(y-F)^2] / \partial F = -(y - F)$$

This is the derivative: 'which direction and how far should  $F$  move to reduce the loss?'

3

### The Negative Gradient IS the Residual

$$-\partial L / \partial F = y - F = \text{residual!}$$

When we fit a tree on  $(y - F_{\text{prev}})$ , we are fitting the NEGATIVE GRADIENT of the loss.  
This is gradient DESCENT on the loss function

4

### Update Rule = Gradient Descent Step

$$\begin{aligned} F_{\text{new}} &= F_{\text{old}} + \eta \times (-\text{gradient}) \\ &= F_{\text{old}} + \eta \times (y - F_{\text{old}}) \\ &= F_{\text{old}} + \eta \times \text{residual} \end{aligned}$$

$\eta$  is the learning rate — step size in gradient descent.

## Why Does This Matter?

For MSE loss, the gradient is just the residual — simple and intuitive.

But the 'Gradient' in the name means we can swap the loss function:

- Log-loss  $\rightarrow$  classification
- MAE  $\rightarrow$  robust to outliers
- Custom loss  $\rightarrow$  business-specific

The algorithm stays the same. Only the pseudo-residuals change.

XGBoost uses second-order gradients (Hessian) for

## Concrete Numbers

Row 1:  $y=15k$ ,  $F_0=23k$

Loss:  $\frac{1}{2}(15-23)^2 = 32$

Gradient:  $-(15-23) = +8 \rightarrow F$  is too high, push DOWN

Residual:  $15-23 = -8 \checkmark$  Same!

$F_1 = 23 + 0.1 \times (-8) = 22.2k$

New loss:  $\frac{1}{2}(15-22.2)^2 = 25.9 < 32 \checkmark$

Summary: Fitting trees on residuals = gradient descent in function space. The learning rate  $\eta$  is the step size. More trees = more steps toward minimum loss.



# Gradient Boosting: Learning Rate & Overfitting

The critical tradeoff: speed vs. generalization

## Learning Rate ( $\eta$ ) Explained

$\eta = 1.0$   
(too fast)

Each tree corrects 100% of the error.  
Aggressive, can overfit training data.

✗ Too risky

$\eta = 0.1$   
(recommended)

Each tree corrects only 10%. Needs  
more trees but generalizes better.

✓ Use this

$\eta = 0.01$   
(too slow)

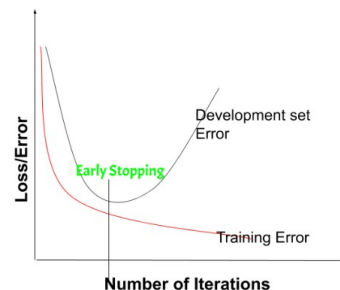
Each tree corrects only 1%. Needs  
thousands of trees. Very slow.

⚠ Too slow

Rule: Lower  $\eta \rightarrow$  need more  $n_{\text{estimators}}$ . Use early stopping to find the sweet spot automatically.

## Early Stopping (Essential!)

Train error always goes down.  
Validation error goes down... then up!  
Stop when validation error starts rising.



```
# XGBoost with early stopping
xgb = XGBClassifier(
    n_estimators=1000,
    learning_rate=0.05,
    early_stopping_rounds=20
)
xgb.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    verbose=False
```

# XGBoost & LightGBM: Why Industry Uses Them

Gradient Boosting + regularization + engineering = state of the art

## Gradient Boost (sklearn)

The foundation.  
Simple, educational, slow on large data.

✓ Easy to understand

✗ Slow, no GPU

```
GradientBoostingClassifier(  
    n_estimators=100,  
    learning_rate=0.1,  
    max_depth=3  
)
```

## XGBoost

Adds L1+L2 regularization, second-order  
gradients, parallel splits, pruning.

✓ Fast, accurate, handles missing data

✗ Many hyperparams

```
XGBClassifier(  
    n_estimators=300,  
    learning_rate=0.05,  
    max_depth=5,  
    reg_alpha=0.1, # L1  
    reg_lambda=1.0 # L2  
)
```

## LightGBM

Leaf-wise growth instead of level-wise.  
Histogram-based splits. Fastest.

✓ Often substantially faster than XGBoost on  
large datasets

✗ Can overfit on small data

```
LGBMClassifier(  
    n_estimators=300,  
    learning_rate=0.05,  
    num_leaves=31,  
    min_child_samples=20  
)
```

Industry default: Try XGBoost or LightGBM first. They outperform sklearn's GradientBoosting on most real datasets.

# XGBoost: The Objective Function — Full Math

Loss + Regularization — why this makes XGBoost better than plain Gradient Boosting

$$\text{Obj}(\theta) = \sum_i L(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

Training Loss (fit the data) + Regularization (keep the model simple)

$$\sum_i L(y_i, \hat{y}_i)$$

Training Loss

Sum over all N training examples of the loss between true label  $y_i$  and prediction  $\hat{y}_i$ .

For regression:  $\text{MSE} = \sum (y_i - \hat{y}_i)^2$

For classification:  $\text{Log-loss} = -\sum [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$

**Purpose:**

*Pushes predictions toward the truth. Lower = better fit.*

$$\sum_k \Omega(f_k)$$

Regularization Term

Sum over all K trees. For each tree:

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_j w_j^2$$

$\gamma$  = penalty per leaf (T = number of leaves)

$\lambda$  = L2 penalty on leaf weights  $w_j$

$\alpha$  = L1 penalty (makes some weights exactly 0)

**Purpose:**

*Penalises complexity. Forces trees to be smaller and weights to be smaller.*

With regularization: XGBoost builds fewer, simpler trees that generalise better.

$\gamma > 0$  forces pruning.  $\lambda > 0$  shrinks leaf scores toward zero — like weight

Plain Gradient Boosting has NO  $\Omega$  term — it's just  $\sum L(y_i, \hat{y}_i)$ .

With no penalty on complexity, it will keep growing trees until it overfits.

# Bagging vs Boosting: Why Does the Distinction Matter?

Two ensemble strategies solving two completely different problems

## Analogy: Two Ways to Build a Better Hiring Process

Bagging: Ask 100 interviewers to evaluate the same candidate independently and vote. Good because no single interviewer's bias dominates.

Boosting: Run 5 rounds of interviews. Each round focuses on the specific gaps the last round missed. Good because each round builds on the last.

### Bagging / Random Forest

 **Solves: OVERFITTING (high variance)**

#### Use when you see these symptoms:

- Your model scores 99% train, 75% test
- Small changes to training data flip predictions
- You have lots of noisy or mislabelled data

#### Prescription:

**Random Forest: train 100 uncorrelated trees, average their votes.**

**Train: 92% Test: 89% ← the gap closes**

### Boosting (AdaBoost / XGBoost)

 **Solves: UNDERFITTING (high bias)**

#### Use when you see these symptoms:

- Train accuracy is 75%, test is 74% — both bad
- Adding more features doesn't help much
- You need every % of accuracy you can get

#### Prescription:

**XGBoost: 500 sequential trees, each one correcting the previous one's mistakes.**

**Train: 95% Test: 91% ← higher overall**

Next slide: the full head-to-head comparison table so you always know which one to reach for.

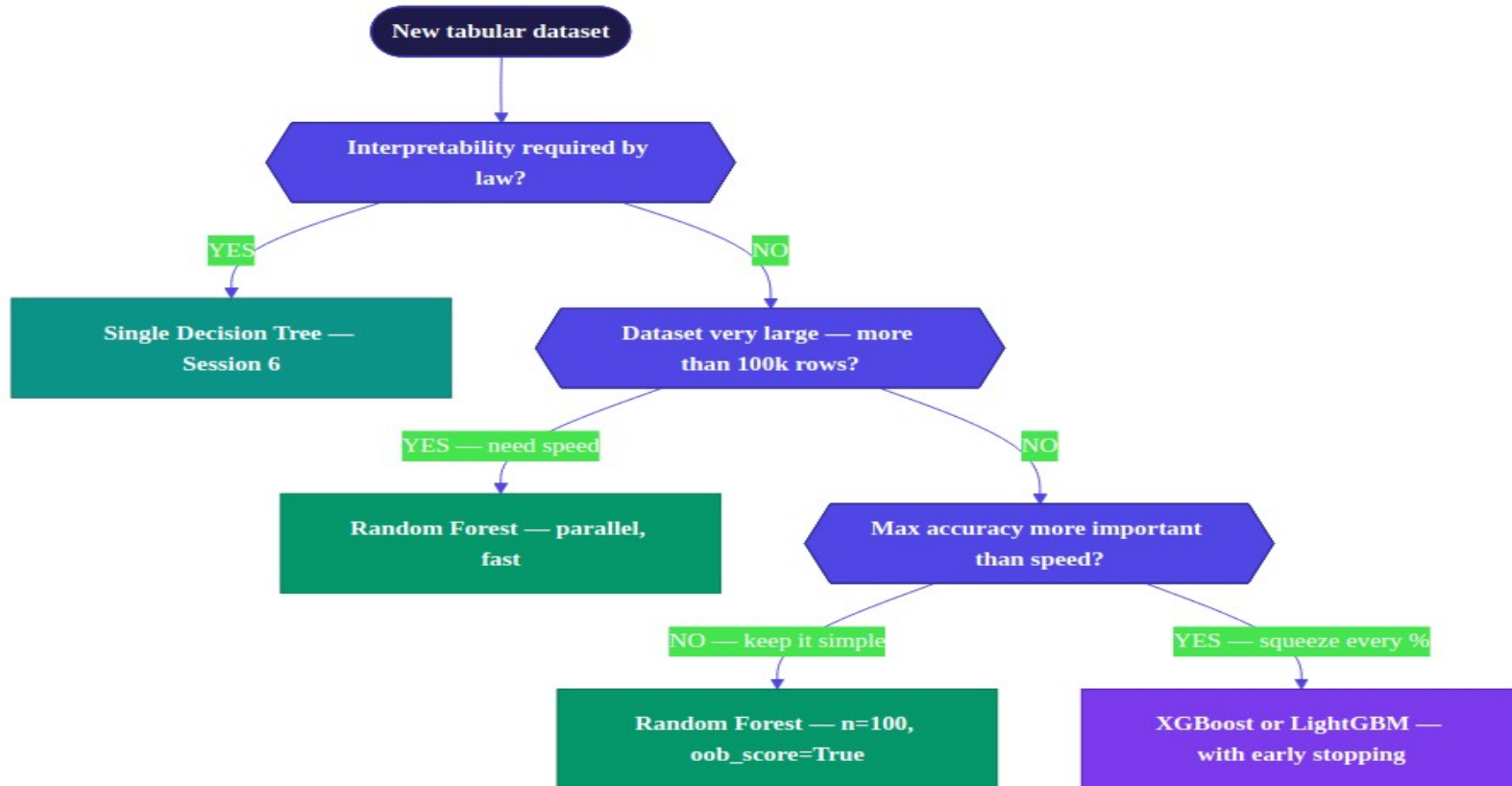
# Bagging vs Boosting: Complete Comparison

Two strategies, two problems they solve

|                  | Bagging / Random Forest          | Boosting (AdaBoost / XGBoost)                                                    |
|------------------|----------------------------------|----------------------------------------------------------------------------------|
| Training         | Parallel — trees are independent | Sequential — each tree depends on previous                                       |
| What it fixes    | High Variance (overfit)          | High Bias (underfit)                                                             |
| Base learner     | Full deep trees (low bias)       | Shallow trees / stumps (weak learners)                                           |
| Weighting        | Equal vote for all trees         | AdaBoost: weighted by accuracy ( $\alpha$ ); GB: additive steps scaled by $\eta$ |
| Overfitting risk | Low (averaging reduces it)       | Higher — needs regularization / early stop                                       |
| Speed            | Fast (parallel)                  | Slower (sequential)                                                              |
| Interpretability | Feature importance               | Feature importance + SHAP                                                        |
| Missing values   | Needs imputation                 | XGBoost handles natively                                                         |
| When to use      | Noisy data, quick baseline       | When you need max accuracy                                                       |

# Which Method Should I Use? — Decision Framework

WHY — Business Intuition



Noisy data:  
Random Forest

Max accuracy:  
XGBoost /  
LightGBM

5-min baseline:  
Random Forest

Need explanation:  
Decision Tree →  
then RF

# Code: Complete Ensemble Workflow

Load data → fit RF & XGBoost → compare → interpret

```
from sklearn.ensemble import (
    RandomForestClassifier,
    GradientBoostingClassifier
)
from xgboost import XGBClassifier
from sklearn.model_selection import cross_val_score
from sklearn.datasets import load_breast_cancer
import numpy as np
import pandas as pd

# 1. Load data
X, y = load_breast_cancer(return_X_y=True)
feature_names = load_breast_cancer().feature_names
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# 2. Models
rf = RandomForestClassifier(
    n_estimators=100, oob_score=True,
    n_jobs=-1, random_state=42)

xgb = XGBClassifier(
    n_estimators=200, learning_rate=0.05,
    max_depth=4, eval_metric='logloss',
    random_state=42)

# 3. Cross-validate (5-fold)
for name, model in [("RF", rf), ("XGB", xgb)]:
    scores = cross_val_score(
        model, X, y, cv=5, scoring='roc_auc')
    print(f"{name}: {scores.mean():.3f} ± "
          f"{scores.std():.3f}")
```

## What Each Step Does

### `cross_val_score`

Always use CV — never just train/test split for ensemble comparison

### `oob_score=True`

Free validation using the 37% of data not seen by each tree

### `n_jobs=-1`

Use all CPU cores — RF is embarrassingly parallel

### `learning_rate=0.05`

Low LR + many trees = better generalization for XGBoost

### `feature_importances_`

Which features matter most? Use for business explanation

# Hyperparameter Tuning: What to Adjust and Why

Ordered by importance — start with the most impactful

| Random Forest                          |                                                                      |      |
|----------------------------------------|----------------------------------------------------------------------|------|
| <b>n_estimators</b><br>default: 100    | ↑ more stable; test error typically plateaus. Cost: slower training. | HIGH |
| <b>max_features</b><br>default: "sqrt" | "sqrt" for classification, "log2" or 0.3 for regression.             | HIGH |
| <b>max_depth</b><br>default: None      | Limit if overfitting. Start unconstrained.                           | MED  |
| <b>min_sample_leaf</b><br>default: 1   | ↑ prevents tiny leaves, reduces overfit on small data.               | MED  |
| <b>class_weight</b><br>default: None   | "balanced" for imbalanced classes.                                   | HIGH |

| XGBoost / LightGBM                                       |                                                       |      |
|----------------------------------------------------------|-------------------------------------------------------|------|
| <b>learning_rate (<math>\eta</math>)</b><br>default: 0.3 | Use 0.05-0.1. Lower = better, needs more trees.       | HIGH |
| <b>n_estimators</b><br>default: 100                      | Set to 1000, use early stopping to find best.         | HIGH |
| <b>max_depth</b><br>default: 6                           | 3-6 typical. Deeper → more complex → overfit risk.    | HIGH |
| <b>reg_alpha (L1)</b><br>default: 0                      | Sparse solutions — use when many irrelevant features. | MED  |
| <b>reg_lambda (L2)</b><br>default: 1                     | Smooth solutions — increase if overfitting.           | MED  |
| <b>subsample</b><br>default: 1.0                         | 0.8 = use 80% of rows per tree. Reduces overfit.      | MED  |

Start: keep defaults except `n_estimators=300`, `learning_rate=0.05`, early stopping. Only tune `max_depth` if still overfitting.



# Common Mistakes — and How to Fix Them

*What goes wrong in practice, and what to do instead*

1

## ✗ Using train accuracy to compare RF vs XGBoost

✓ Prefer cross-validation over a single train/test split. RF and XGBoost both overfit training data — CV gives a more reliable estimate of generalisation.

2

## ✗ Setting `n_estimators` too low (e.g. 10 trees)

✓ Start with 100–500. More trees = more stable predictions. Adding trees rarely hurts accuracy — the main cost is training time.

3

## ✗ Using boosting on very noisy labels

✓ Boosting amplifies noise — it focuses more and more on 'hard' examples that may just be mislabeled. Use RF on noisy data.

4

## ✗ Not using early stopping with XGBoost

✓ Set `n_estimators=1000`, `early_stopping_rounds=20`. Without it, XGBoost will silently overfit to training data.

5

## ✗ Confusing feature importance between RF and XGBoost

✓ RF MDI is biased toward high-cardinality features. XGBoost importance has multiple types (gain, cover, weight). Use SHAP for reliable interpretation.

6

## ✗ Thinking more trees always means slower inference

✓ Inference is generally fast even with 500 trees, since they run in parallel. For latency-sensitive production systems, measure prediction time explicitly.

# Connecting to Business: What Do You Tell a Stakeholder?

*The WHY lens — translating model results to business language*

Why? Business

*Scenario: You trained a Random Forest on BankEU loan data. How do you present results to the credit committee?*

## **Q: Why should we trust 100 trees more than 1?**

Each tree makes different errors on different data. When 80 out of 100 trees agree an applicant should be rejected, that's a very robust signal — much stronger than any single model.

## **Q: What's the most important factor in rejections?**

Feature importance shows Income accounts for 55% of the model's total impurity reduction. Debt ratio accounts for 30%. Age contributes little. This is actionable — focus underwriting rules on income verification.

## **Q: How accurate is the model?**

On held-out data (OOB), the model correctly classifies 87% of applicants. More importantly: it catches 92% of future defaults (recall on the bad class).

## **Q: Can you explain why applicant 42 was rejected?**

This requires SHAP (Session 8). For now: RF gives you which features matter most, not why for individual predictions.

# Session 7 Summary: What You Now Know

*Six things that should be clear before you leave*

1

Ensembles work because uncorrelated errors cancel out — this is the Wisdom of Crowds, not magic.

2

Bagging trains trees in parallel on bootstrap samples. It reduces variance — the instability of single trees.

3

Random Forest adds random feature subsets at each split. This forces tree diversity and prevents correlated errors.

4

Boosting trains sequentially, each model fixing the errors of the previous. It reduces bias — weak learners become strong.

5

XGBoost/LightGBM are gradient boosting + regularization + engineering. Use them with early stopping for best results.

6

Start with Random Forest as your baseline. Switch to XGBoost when you need to squeeze out more accuracy.